

GUIDE PRATIQUE

Rédiger un Cahier des Charges

Selon le template officiel La Plateforme_

 **Objectif : Rédiger un CDC complet en suivant la structure officielle DWWM**

 Temps estimé : 3-4 heures en groupe

1. Vue d'ensemble du template

Le template La Plateforme_ structure le CDC en 4 grandes parties :

1	Synopsis	Description générale, besoin, contexte
2	CDC Frontend	Fonctionnel (US) + Non-fonctionnel (routes, sécu, deploy)
3	CDC Backend	Fonctionnel (besoin) + Non-fonctionnel (stack, endpoints, RGPD, logs)
4	Annexes	Tous les diagrammes (MCD, UML, maquettes...)



Ce template sépare clairement Frontend et Backend, c'est une approche professionnelle qui facilite le travail en équipe !

2. Synopsis

Le synopsis est ton introduction. En quelques paragraphes, tu dois donner une vision claire du projet.

Ce que tu dois y mettre

- Description de l'application (qu'est-ce que c'est ?)
- Le besoin de la cible (quel problème résout-elle ?)
- Le contexte d'utilisation (où, quand, comment sera-t-elle utilisée ?)



Synopsis - Réservation de Salles

L'application de réservation de salles permet aux collaborateurs de TechSpace Solutions de réserver des espaces de réunion de manière autonome.

Besoin : Actuellement, les réservations se font par email, ce qui génère des conflits et une perte de temps.

Contexte : L'application sera utilisée quotidiennement par environ 50 employés, principalement sur desktop pendant les heures de bureau (8h-19h).

3. Cahier des Charges Frontend

3.1 Partie Fonctionnelle

Cette section décrit CE QUE fait l'application côté utilisateur.

Besoin

Décris le besoin utilisateur de manière claire et concise.

User Stories & Epics

Reprends tes User Stories et organise-les par Epic. Pour chaque US, ajoute les **critères de validation détaillés** (c'est important !).

User Story avec critères de validation

US-01 : En tant qu'utilisateur, je veux me connecter afin d'accéder à mes réservations.

Critères de validation :

- ✓ Le formulaire contient email + mot de passe
- ✓ Message d'erreur si identifiants incorrects
- ✓ Redirection vers dashboard si succès
- ✓ Token JWT stocké dans localStorage
- ✓ Session expire après 24h

3.2 Partie Non-Fonctionnelle (Technique)

Cette section décrit COMMENT l'application est construite techniquement.

Routes Frontend

Liste toutes les routes de ton application React :

Route	Composant	Auth	Description
/login	Login	✗	Page de connexion
/register	Register	✗	Page d'inscription
/dashboard	Dashboard	✓	Tableau de bord
/rooms	Rooms	✓	Liste des salles
/reservations	MyReservations	✓	Mes réservations

Contraintes de Sécurité Frontend

- Routes protégées (PrivateRoute) pour les pages authentifiées
- Token JWT stocké de manière sécurisée
- Validation des inputs côté client
- Protection XSS (pas de dangerouslySetInnerHTML)

Déploiement Frontend

- Plateforme : Vercel / Netlify / GitHub Pages
- Build : npm run build (Vite)
- Variables d'environnement : VITE_API_URL

4. Cahier des Charges Backend

4.1 Partie Fonctionnelle

Besoin

Réponds à ces questions importantes :

- Qui consomme le backend ? (Frontend React, app mobile, autre API ?)
- Combien d'utilisateurs simultanés ? (estimation réaliste)
- Pics de charge prévisibles ? (ex: lundi matin = + de réservations)

Besoin Backend - Réservation de Salles

Consommateur : Application React (SPA)

Utilisateurs simultanés : ~20 max en période normale

Pic de charge : Lundi 8h-10h (~50 requêtes/min)

Disponibilité requise : 99% (heures ouvrées)

4.2 Partie Non-Fonctionnelle (Technique)

Stack Technique

Runtime	Node.js v18+
Framework	Express.js v4
Base de données	MySQL v8
Authentification	JWT + bcrypt
ORM	Aucun (requêtes SQL natives)

MCD / Schéma UML

Intègre ton MCD ici (image ou diagramme). Décris chaque entité et ses relations.

Endpoints API

Pour chaque route, précise : méthode, URL, description, codes de statut, et DTO (structure des données).

Méthode	Endpoint	Description	Status
POST	/api/auth/register	Inscription	201, 400, 409
POST	/api/auth/login	Connexion	200, 401
GET	/api/auth/me	Profil	200, 401
GET	/api/rooms	Liste salles	200
GET	/api/reservations	Mes réservations	200, 401
POST	/api/reservations	Créer réservation	201, 400, 409
DELETE	/api/reservations/:id	Annuler	200, 403, 404

Exemple de DTO (Data Transfer Object)

POST /api/reservations

Request Body :

```
{
  "room_id": 1,
  "date": "2025-02-10",
  "start_time": "09:00",
  "end_time": "10:00",
  "title": "Réunion équipe"
}
```

Response 201 :

```
{
  "id": 42,
  "message": "Réservation créée"
}
```

Contraintes de Sécurité Backend

Authentification	JWT (JSON Web Token) avec expiration 24h
Hachage MDP	bcrypt avec salt (10 rounds)
RBAC	Contrôle d'accès basé sur les rôles (si applicable)
Validation	Validation des entrées côté serveur (email, formats)
SQL Injection	Requêtes paramétrées (prepared statements)
CORS	Origine autorisée : http://localhost:5173

RGPD

Le RGPD (Règlement Général sur la Protection des Données) impose des règles sur les données personnelles :

- PII (Personally Identifiable Information) : Limiter la collecte au strict nécessaire
- Anonymisation : Possibilité d'anonymiser les données sur demande
- Logs : Ne pas logger d'informations sensibles (IP, identifiants, mots de passe)
- Droit à l'oubli : Possibilité de supprimer un compte et ses données

 **Pour ce projet, précise : quelles données personnelles tu collectes (email, nom) et comment tu les protèges.**

Logging

Définis ta stratégie de logs :

	DEBUG	Détails techniques (dev uniquement)
i	INFO	Actions normales (connexion, réservation)
	WARNING	Comportements suspects
	ERROR	Erreurs applicatives

Performance (si applicable)

Pour un projet avancé, tu peux préciser :

- RPS (Requests Per Second) : combien de requêtes/seconde supportées
- Latence : temps de réponse moyen attendu (ex: < 200ms)
- Outils de test : Apache Benchmark, k6, Locust...

 **Pour ce projet DWWM, une section performance simple suffit. Les tests de charge sont un bonus !**

5. Annexe des diagrammes

Regroupe ici tous tes diagrammes :

- MCD (Modèle Conceptuel de Données)
- MLD (Modèle Logique de Données)
- Diagramme de cas d'utilisation (Use Case)
- Diagramme de séquence (si pertinent)
- Maquettes / Wireframes

💡 **Utilise draw.io, Excalidraw ou MySQL Workbench pour créer tes diagrammes. Exporte-les en PNG ou PDF.**

6. Checklist de validation

Avant de rendre ton CDC, vérifie que tu as bien complété chaque section :

☐	Synopsis	Description, besoin, contexte
☐	CDC Frontend - Fonctionnel	User Stories avec critères de validation
☐	CDC Frontend - Non-fonctionnel	Routes, sécurité, déploiement
☐	CDC Backend - Fonctionnel	Besoin, utilisateurs, charge
☐	CDC Backend - Stack	Technologies utilisées
☐	CDC Backend - Endpoints	Toutes les routes avec status codes
☐	CDC Backend - Sécurité	Auth, validation, CORS
☐	CDC Backend - RGPD	Données personnelles, logs
☐	Annexes	MCD, diagrammes, maquettes

7. Ressources pour aller plus loin

Vidéos recommandées

Grafikart - API REST

Comprendre les principes REST, les codes HTTP, et la structure des endpoints.

 <https://grafikart.fr/tutoriels/api-rest>

OpenClassrooms - RGPD

Cours gratuit sur la protection des données personnelles.

 <https://openclassrooms.com/fr/courses/6175906-appliquez-le-rgpd>

Documentation

HTTP Status Codes

Référence complète des codes de statut HTTP (200, 201, 400, 401, 404, 500...).

 <https://developer.mozilla.org/fr/docs/Web/HTTP/Status>

JWT.io

Comprendre et débbugger les tokens JWT.

 <https://jwt.io/introduction>